

Ex.No.3	Wireshark
Date	

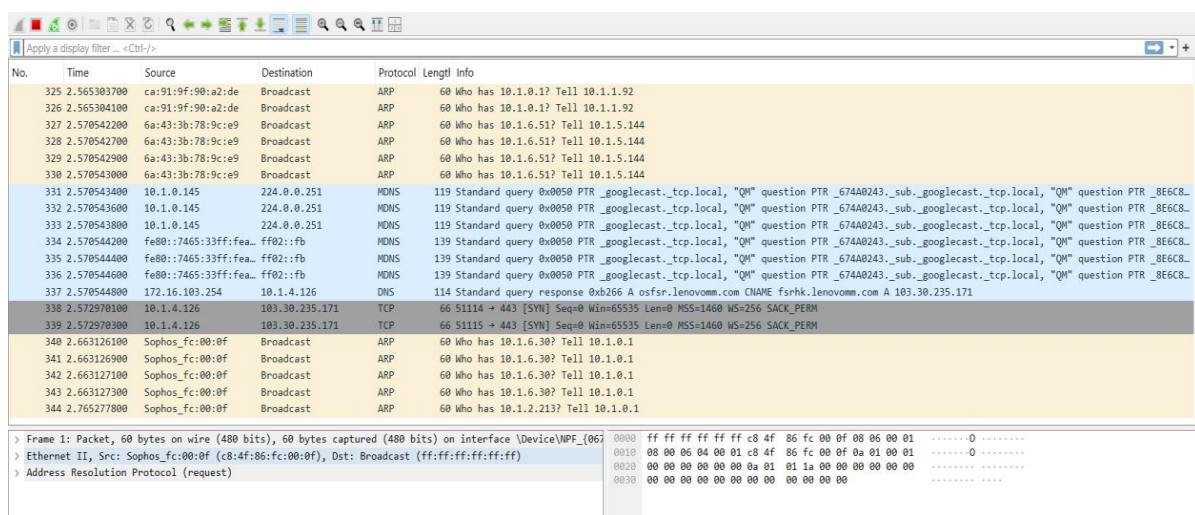
Password Capturing

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

So, here we will see how we can capture the password using the Wireshark network capture analyzer. and see the outputs of the following steps.

Step 1: First of all, open your Wireshark tool in your window or in Linux virtual machine. and start capturing the network. suppose I am capturing my wireless fidelity.



Step 2: After starting the packet capturing we will go to the website and login the credential on that website as you can see in the image.

Step 3: Now after completing the login credential we will go and capture the password in Wireshark. for that we have to use some filter that helps to find the login credential through the packet capturing.

Step 4: Wireshark has captured some packets but we specifically looking for HTTP packets. so in the display filter bar we use some command to find all the captured HTTP packets. as you can see in the below image the green bar where we apply the filter.
http

No.	Destination	Protocol	Length	Info
1	127.0.0.1	HTTP	756	GET / HTTP/1.1
2	127.0.0.1	HTTP	44	HTTP/1.0 200 OK (text/html)
3	127.0.0.1	HTTP	723	GET / HTTP/1.1
4	127.0.0.1	HTTP	44	HTTP/1.0 200 OK (text/html)
5	127.0.0.1	HTTP	649	GET /favicon.ico HTTP/1.1
6	127.0.0.1	HTTP	649	GET /favicon.ico HTTP/1.1
7	127.0.0.1	HTTP	936	POST /login HTTP/1.1 (application/x-www-form-urlencoded)
8	127.0.0.1	HTTP	44	HTTP/1.0 200 OK (text/html)
9	127.0.0.1	HTTP	654	GET /favicon.ico HTTP/1.1
10	127.0.0.1	HTTP	654	GET /favicon.ico HTTP/1.1

```

> Frame 707: Packet, 756 bytes on wire (6048 bits), 756 bytes captured (6048 bits) on interface \Device\NPF...
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 61231, Dst Port: 8080, Seq: 1, Ack: 1, Len: 712
> Hypertext Transfer Protocol
0000  02 00 00 00 45 00 02 f0 27 e4 40 00 00 06 d2 21  ....E-
0010  7f 00 00 01 7f 00 00 01 ef 2f 1f 40 3f 68 4d bd  ....
0020  b2 b7 4d ad 50 18 00 ff 4d 9a 00 00 47 45 54 20  ...M.P-
0030  2f 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74  / HTTP
0040  3a 20 31 32 37 2e 30 2e 30 2e 31 3a 38 30 30 30  : 127.1
0050  0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65  ..Conn
0060  65 70 2d 61 6c 69 76 65 0d 0a 73 65 63 2d 63 68  ep-ali
0070  2d 75 61 3a 20 22 43 68 72 6f 6d 69 75 6d 22 3b  -ua: "I
0080  76 3d 22 31 35 32 22 2c 20 22 4e 6f 74 3f 41 5f  v="152/
0090  42 72 61 6e 64 22 3b 76 3d 22 32 34 22 2c 20 22  Brand"
00a0  4d 69 63 72 6f 73 6f 66 74 20 45 64 67 65 22 3b  Microso
00b0  76 3d 22 31 35 32 22 0d 0a 73 65 63 2d 63 68 2d  v="152/
00c0  75 61 2d 6d 6f 62 69 6c 65 3a 20 3f 30 0d 0a 73  ua-mob:
00d0  65 63 2d 63 68 2d 75 61 2d 70 6c 61 74 66 6f 72  ec-ch-i

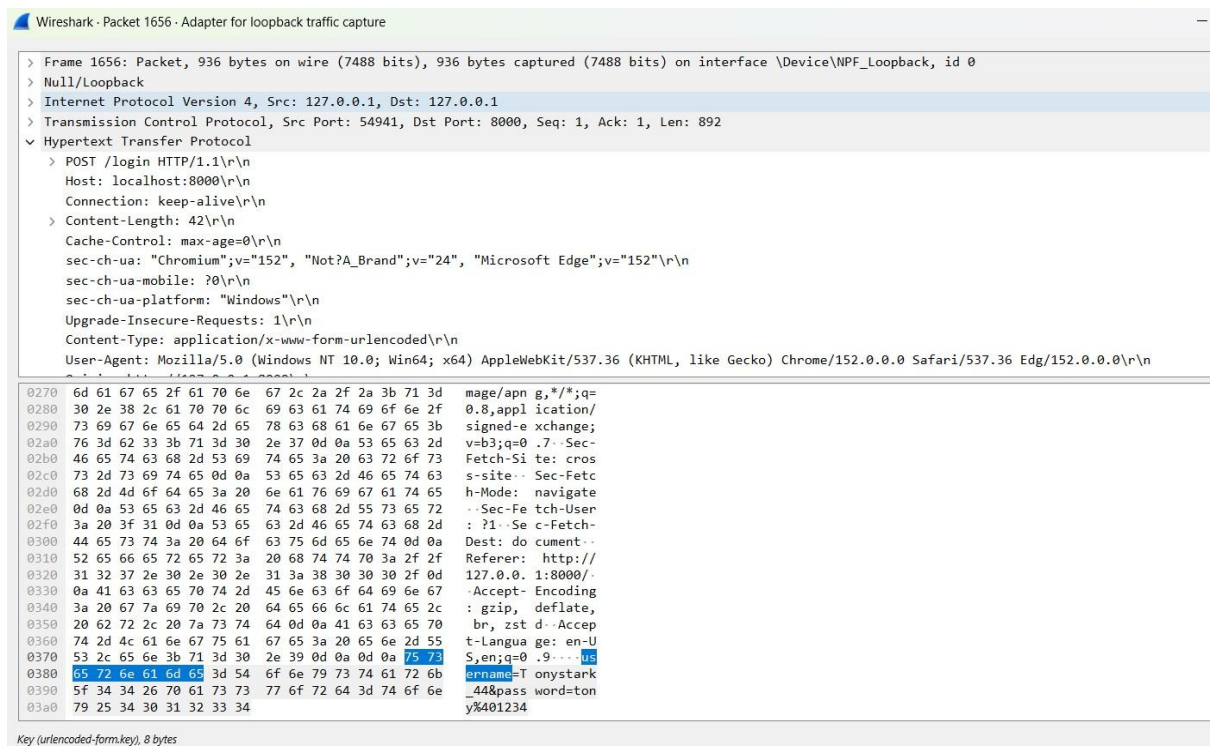
```

Step 5: So there are some HTTP packets are captured but we specifically looking for form data that the user submitted to the website. for that, we have a separate filter

As we know that there are main two methods used for submitting form data from web pages like login forms to the server. the methods are-

- GET
- POST

Step 6: So firstly for knowing the credential we use the first method and apply the filter for the GET methods as you can see below.
`http.request.method == "GET"`



The image shows a Wireshark packet capture of an HTTP GET request. The packet list on the left shows a packet of 936 bytes on the wire. The packet details pane on the right shows the following structure:

- Frame 1656: Packet, 936 bytes on wire (7488 bits), 936 bytes captured (7488 bits) on interface \Device\NPF_{...}, id 0
- Null/Loopback
- Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- Transmission Control Protocol, Src Port: 54941, Dst Port: 8000, Seq: 1, Ack: 1, Len: 892
- Hypertext Transfer Protocol
 - POST /login HTTP/1.1\r\n
 - Host: localhost:8000\r\n
 - Connection: keep-alive\r\n
 - Content-Length: 42\r\n
 - Cache-Control: max-age=0\r\n
 - sec-ch-ua: "Chromium";v="152", "Not?A_Brand";v="24", "Microsoft Edge";v="152"\r\n
 - sec-ch-ua-mobile: ?0\r\n
 - sec-ch-ua-platform: "Windows"\r\n
 - Upgrade-Insecure-Requests: 1\r\n
 - Content-Type: application/x-www-form-urlencoded\r\n
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36 Edg/152.0.0.0\r\n

The packet bytes pane at the bottom shows the raw data of the request, including the headers and the body. The body contains the following data:

```
image/apng,/*,q=0.8,application/signed-exchange;v=b3;q=0.7\r\nSec-Fetch-Site: cross-site\r\nSec-Fetch-Mode: navigate\r\nSec-Fetch-User: ?1\r\nSec-Fetch-Dest: document\r\nReferer: http://127.0.0.1:8000/\r\nAccept-Encoding: gzip, deflate, br, zstd\r\nAccept-Language: en-US,en;q=0.9\r\nname=tonystark_44&password=tony%401234
```

GET method

As you can see in the image there are two packets where the login page was requested with a GET request as well, but there is no form data submitted with a GET request.

Step 7: Now after checking the GET method if we didn't find the form data, then we will try the POST method for that we will apply the filter on Wireshark as you can see.
`http.request.method == "POST"`

Rubrics:

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1.GitHub Activity & Submission Regularity	3	
2.Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

Result:

Wireshark successfully captured and filtered HTTP packets to identify the login form data transmitted during the experiment.

The username and password fields were observed in the HTTP POST request, demonstrating the security risk of transmitting credentials over unencrypted HTTP.